TRƯỜNG ĐẠI HỌC FPT

BÁO CÁO DỰ ÁN

THIẾT KẾ RTL BỘ NHỚ FIFO BẤT ĐỒNG BỘ

(ASYNCHRONOUS FIFO)

Môn học: FAP201 - FPGA Programming

Giảng viên hướng dẫn: Phạm Thế Vinh

Lớp: IC2007 **- Học kỳ:** FA26

Danh sách nhóm thực hiện

1. Âu Quốc Kim Sinh SE201166
2. Trần Minh Đức SE204259
3. Trần Đỗ Minh Anh SE204631
4. Lưu Ngọc Hoàn Kim SE205028
5. Lưu Ngọc Hoàn Mỹ SE204355
6. Lưu Hoàng Trọng Hiếu SE204809

Thành phố Hồ Chí Minh, 2026

MỤC LỤC

I. Tổng Quan Đề Tài

1. Mục tiêu
2. Nội dung đề tài

II. Cơ Sở Lý Thuyết

1. Nguyên lý FIFO bất đồng bộ
2. Clock Domain Crossing và metastability
3. Gray Code và bộ đồng bộ hai tầng
4. Điều kiện Full, Empty và ngưỡng cảnh báo

III. Thiết Kế RTL Cho Module Asynchronous Fifo

1. Tổng quan kiến trúc hệ thống
2. Module asynchronous_fifo
3. Module fifo_mem
4. Module synchronizer
5. Module wptr_handler
6. Module rptr_handler

IV. Mô Phỏng Và Kiểm Thử

1. Môi trường và thông số testbench
2. Các kịch bản kiểm thử
3. Phân tích hạn chế của testbench
4. Hướng dẫn mô phỏng và ghi nhận kết quả

V. Kết Luận

VI. Tài Liệu Tham Khảo

PHỤ LỤC – CÁC ĐOẠN RTL TIÊU BIỂU

I. Tổng Quan Đề Tài

1. Mục tiêu

Đề tài hướng đến thiết kế bộ đệm First-In First-Out (FIFO) bất đồng bộ bằng Verilog HDL, cho phép bên ghi và bên đọc hoạt động theo hai xung nhịp độc lập. Trọng tâm là tổ chức đường dữ liệu, quản lý con trỏ đọc/ghi và giảm rủi ro khi truyền thông tin điều khiển qua ranh giới miền clock (Clock Domain Crossing – CDC).

– Xây dựng cấu trúc RTL phân cấp, có thể điều chỉnh độ rộng dữ liệu và dung lượng FIFO bằng parameter.

– Sử dụng con trỏ nhị phân để truy cập bộ nhớ và con trỏ Gray để trao đổi trạng thái giữa hai miền clock.

– Đồng bộ các con trỏ Gray bằng hai tầng flip-flop ở miền clock đích.

– Tạo cờ full/empty và almost_full/almost_empty nhằm điều khiển ghi, đọc an toàn.

– Xây dựng testbench SystemVerilog có bộ nhớ tham chiếu và so sánh dữ liệu đầu ra.

2. Nội dung đề tài

Đầu tiên, nghiên cứu cơ chế FIFO và vấn đề CDC. Tiếp theo, xây dựng các module bộ nhớ hai cổng, bộ đồng bộ, bộ quản lý con trỏ ghi và con trỏ đọc. Module top-level tích hợp các khối này thành một giao diện FIFO thống nhất. Cuối cùng, sử dụng testbench với hai xung nhịp khác nhau để kiểm tra thứ tự dữ liệu và trạng thái FIFO.

Phạm vi mã nguồn hiện tại tập trung vào RTL và mô phỏng chức năng; chưa có file ràng buộc chân FPGA, báo cáo tổng hợp tài nguyên, kiểm tra timing hoặc ảnh waveform đã chạy. Vì vậy không khẳng định kết quả thực nghiệm hay việc triển khai trên kit phần cứng.

II. Cơ Sở Lý Thuyết

1. Nguyên lý FIFO bất đồng bộ

FIFO lưu dữ liệu theo nguyên tắc phần tử ghi trước được đọc trước. Trong thiết kế bất đồng bộ, thao tác ghi được kích hoạt bởi wclk, thao tác đọc bởi rclk; hai xung nhịp có thể khác tần số và pha. FIFO tạo vùng đệm để hai khối có tốc độ xử lý khác nhau không phải giao tiếp trực tiếp từng phần tử.

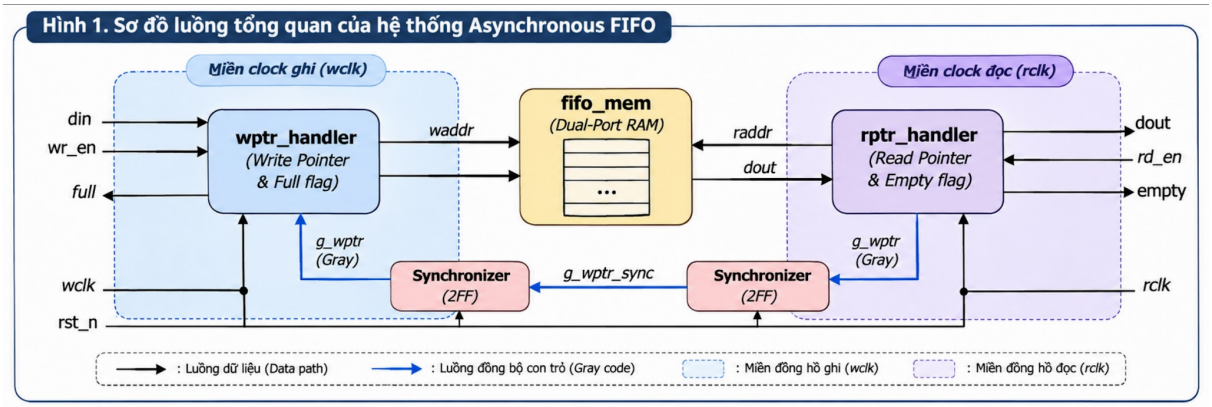
Với PTR_WIDTH = P, bộ nhớ có DEPTH = 2^P ô; mỗi ô có DATA_WIDTH bit. Con trỏ sử dụng P+1 bit, trong đó P bit thấp làm địa chỉ RAM, bit cao bổ sung thông tin vòng quay để phân biệt đầy và rỗng.

2. Clock Domain Crossing và metastability

Tín hiệu thay đổi gần cạnh lấy mẫu của flip-flop có thể vi phạm thời gian setup/hold, khiến trạng thái đầu ra tạm thời không ổn định (metastability). Đồng bộ hai tầng giúp giảm xác suất trạng thái không ổn định lan sang logic điều khiển, nhưng không thể loại bỏ hoàn toàn rủi ro. Một bus nhị phân nhiều bit có thể đổi nhiều bit cùng lúc; nếu lấy mẫu ở thời điểm chuyển tiếp, giá trị nhận được có thể không đại diện cho một con trỏ hợp lệ.

3. Gray Code và bộ đồng bộ hai tầng

Mã Gray có đặc điểm hai giá trị liên tiếp chỉ khác nhau một bit, phù hợp để truyền con trỏ qua CDC. Công thức chuyển từ nhị phân B sang Gray G: G = B XOR (B >> 1). Tại miền đích, con trỏ Gray được lấy mẫu lần lượt qua stage1 và d_out trong hai chu kỳ clock. Mã Gray chỉ giảm nguy cơ bắt sai nhiều bit; việc hiện thực trên FPGA vẫn cần quan tâm đến độ lệch đường truyền các bit và ràng buộc timing CDC.



Hình 1.

4. Điều kiện Full, Empty và ngưỡng cảnh báo

Empty xảy ra khi con trỏ đọc Gray kế tiếp bằng con trỏ ghi Gray đã đồng bộ. Full xảy ra khi con trỏ ghi Gray kế tiếp bằng con trỏ đọc Gray đã đồng bộ với hai bit cao nhất đảo dấu. Điều kiện được tính từ con trỏ kế tiếp để cập nhật cờ cùng cạnh clock với thay đổi con trỏ.

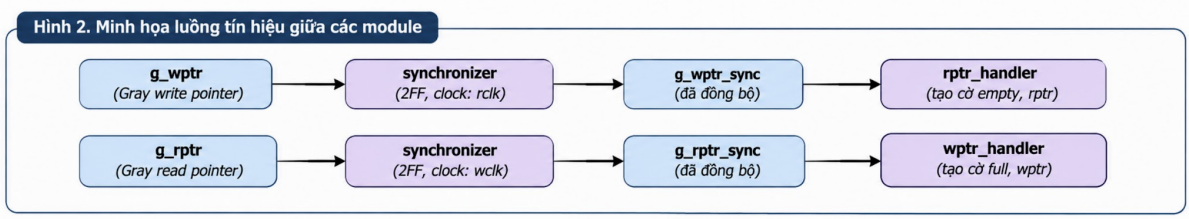
Các cờ almost_full và almost_empty không so sánh Gray trực tiếp: thiết kế chuyển con trỏ Gray đã đồng bộ trở lại nhị phân bằng hàm gray2bin, sau đó tính số phần tử tương đối theo phép trừ modulo 2^(P+1). Giá trị gần đầy được so với DEPTH − ALMOST_FULL_THRESH; gần rỗng được so với ALMOST_EMPTY_THRESH. Do trễ đồng bộ, các cờ có thể phản ánh trạng thái cũ và cần được hiểu là cảnh báo bảo thủ theo miền clock tương ứng.

III. Thiết Kế RTL Cho Module Asynchronous Fifo

1. Tổng quan kiến trúc hệ thống

Hệ thống gồm năm loại module chức năng: top-level asynchronous_fifo, bộ nhớ fifo_mem, bộ đồng bộ synchronizer (2 instance), bộ xử lý con trỏ ghi wptr_handler và bộ xử lý con trỏ đọc rptr_handler. Dữ liệu đi trực tiếp từ data_in đến RAM khi ghi và từ RAM đến data_out khi đọc; chỉ con trỏ Gray được đưa qua hai miền clock.

Thành phần	Chức năng
asynchronous_fifo	Kết nối toàn bộ khối và chặn yêu cầu không hợp lệ
fifo_mem	RAM hai cổng ghi/đọc theo clock riêng
synchronizer ×2	Đồng bộ con trỏ Gray qua hai tầng FF
wptr_handler	Tạo địa chỉ ghi, con trỏ Gray, full/almost_full
rptr_handler	Tạo địa chỉ đọc, con trỏ Gray, empty/almost_empty



Hình 2.

Tham số	Giá trị mặc định RTL	Giá trị testbench
DATA_WIDTH	8	8
PTR_WIDTH	4	3
DEPTH	16	8
ALMOST_FULL_THRESH	2	2
ALMOST_EMPTY_THRESH	2	2
wclk	Từ hệ thống ngoài	50 MHz
rclk	Từ hệ thống ngoài	~14,286 MHz

2. Module asynchronous_fifo

Đây là khối top-level khai báo hai nhóm tín hiệu: miền ghi (wclk, wrst, w_en, data_in, full, almost_full) và miền đọc (rclk, rrst, r_en, data_out, empty, almost_empty). Hai tín hiệu reset là active-high và bất đồng bộ. Module tạo các bus b_wptr, b_rptr, g_wptr, g_rptr và hai con trỏ Gray đã đồng bộ.

Cổng	Chiều	Độ rộng	Chức năng
wclk, wrst	Input	1	Clock và reset miền ghi
w_en	Input	1	Yêu cầu ghi
data_in	Input	DATA_WIDTH	Dữ liệu ghi
full, almost_full	Output	1	Cờ đầy và gần đầy
rclk, rrst	Input	1	Clock và reset miền đọc
r_en	Input	1	Yêu cầu đọc
data_out	Output	DATA_WIDTH	Dữ liệu đọc đồng bộ
empty, almost_empty	Output	1	Cờ rỗng và gần rỗng

Tín hiệu cho phép thực sự của RAM được khóa bởi w_en && !full và r_en && !empty. Nhờ đó, khi full = 1, một yêu cầu ghi không làm thay đổi RAM; khi empty = 1, yêu cầu đọc không cập nhật data_out. Các handler sử dụng cùng điều kiện cho phép để giữ con trỏ nhất quán với thao tác bộ nhớ.

3. Module fifo_mem

fifo_mem là mảng reg [DATA_WIDTH-1:0] mem [0:DEPTH-1]. Khối ghi sử dụng always @(posedge wclk) và nonblocking assignment; khối đọc sử dụng always @(posedge rclk). Dữ liệu đầu ra data_out là thanh ghi, chỉ thay đổi tại cạnh lên rclk khi r_en có hiệu lực.

Cổng	Chiều	Độ rộng	Chức năng
wclk	Input	1	Clock ghi
w_en	Input	1	Cho phép ghi
w_addr	Input	PTR_WIDTH	Địa chỉ ghi
data_in	Input	DATA_WIDTH	Dữ liệu vào
rclk	Input	1	Clock đọc
r_en	Input	1	Cho phép đọc
r_addr	Input	PTR_WIDTH	Địa chỉ đọc
data_out	Output	DATA_WIDTH	Dữ liệu đã đọc

RAM không có reset dữ liệu và data_out cũng không được reset. Vì vậy, trước lần đọc hợp lệ đầu tiên, data_out có thể chưa xác định trong mô phỏng. Đặc tính đọc-ghi đồng thời vào cùng một địa chỉ phụ thuộc cách ánh xạ bộ nhớ khi tổng hợp; không nên suy diễn hành vi collision từ mô phỏng RTL đơn thuần.

4. Module synchronizer

Module synchronizer có tham số WIDTH = 5 mặc định. Khi khởi tạo ở top-level, WIDTH = PTR_WIDTH + 1. Thanh ghi stage1 được gắn thuộc tính (* ASYNC_REG = "TRUE" *), hỗ trợ công cụ nhận diện đường đồng bộ. Ở mỗi cạnh lên clk, stage1 nhận d_in, còn d_out nhận giá trị stage1 của chu kỳ trước. Reset rst active-high bất đồng bộ đặt cả hai về 0.

Cổng	Chiều	Độ rộng	Chức năng
clk	Input	1	Clock miền nhận
rst	Input	1	Reset miền nhận
d_in	Input	WIDTH	Con trỏ Gray miền nguồn
d_out	Output	WIDTH	Con trỏ Gray sau 2FF

Khối này được khởi tạo hai lần: sync_wptr lấy mẫu g_wptr theo rclk, sync_rptr lấy mẫu g_rptr theo wclk. Hai miền reset phải được quản lý nhất quán để tránh diễn giải sai trạng thái FIFO khi một miền đã hoạt động còn miền kia chưa được reset/nhả reset phù hợp.

5. Module wptr_handler

wptr_handler duy trì con trỏ ghi nhị phân b_wptr, con trỏ Gray g_wptr và hai cờ full, almost_full. Địa chỉ ghi w_addr lấy PTR_WIDTH bit thấp của b_wptr. Nếu w_en = 1 và full = 0, b_wptr_next tăng một; ngược lại giữ nguyên. g_wptr_next được tính từ con trỏ nhị phân kế tiếp.

Cổng	Chiều	Độ rộng	Chức năng
wclk, wrst	Input	1	Clock/reset miền ghi
w_en	Input	1	Yêu cầu ghi
g_rptr_sync	Input	PTR_WIDTH+1	Con trỏ đọc Gray đã đồng bộ
b_wptr, g_wptr	Output	PTR_WIDTH+1	Con trỏ ghi nhị phân/Gray

full, almost_full	Output	1	Cờ trạng thái miền ghi
w_addr	Output	PTR_WIDTH	Địa chỉ RAM

Cờ full sử dụng phép so sánh g_wptr_next == {~g_rptr_sync[P:P-1], g_rptr_sync[P-2:0]}. Phép đảo hai bit cao tương ứng với khoảng cách một vòng bộ nhớ trong biểu diễn Gray. Cờ almost_full dùng fill_next = b_wptr_next − gray2bin(g_rptr_sync), so với AF_LEVEL = DEPTH − ALMOST_FULL_THRESH.

Vấn đề kỹ thuật: bộ đồng bộ đưa về con trỏ đọc có độ trễ, vì vậy almost_full có thể được bật sớm hơn trạng thái dung lượng tức thời. Việc thiết kế cờ an toàn cần dựa trên giả thiết reset đồng bộ về trạng thái logic ban đầu và chỉ cho phép ghi khi chưa full.

6. Module rptr_handler

rptr_handler hoạt động đối xứng với miền ghi. b_rptr_next tăng khi r_en && !empty; địa chỉ đọc r_addr lấy các bit thấp. Cờ empty được xác định bằng g_rptr_next == g_wptr_sync. Cờ almost_empty được tính từ fill_next = gray2bin(g_wptr_sync) − b_rptr_next, sau đó so với ALMOST_EMPTY_THRESH.

Cổng	Chiều	Độ rộng	Chức năng
rclk, rrst	Input	1	Clock/reset miền đọc
r_en	Input	1	Yêu cầu đọc
g_wptr_sync	Input	PTR_WIDTH+1	Con trỏ ghi Gray đã đồng bộ
b_rptr, g_rptr	Output	PTR_WIDTH+1	Con trỏ đọc nhị phân/Gray
empty, almost_empty	Output	1	Cờ trạng thái miền đọc
r_addr	Output	PTR_WIDTH	Địa chỉ RAM

Khi reset, empty = 1 và almost_empty = 1; con trỏ đọc nhị phân/Gray về 0. Do con trỏ ghi chỉ xuất hiện ở miền đọc sau hai tầng FF, empty có thể giữ mức 1 thêm một số chu kỳ rclk sau khi phía ghi vừa đưa dữ liệu vào. Đây là độ trễ điều khiển có chủ đích, không đồng nghĩa dữ liệu bị mất.

IV. Mô Phỏng Và Kiểm Thử

1. Môi trường và thông số testbench

File async_fifo_TB.sv khai báo module tb và khởi tạo DUT asynchronous_fifo với DATA_WIDTH = 8, PTR_WIDTH = 3, do đó FIFO có 8 phần tử. wclk đổi mức mỗi 10 ns (chu kỳ 20 ns, 50 MHz); rclk đổi mức mỗi 35 ns (chu kỳ 70 ns, khoảng 14,286 MHz). Reset được giữ ở mức 1 trong 100 ns đầu rồi cùng nhả về 0. Sau đó testbench chờ 50 ns mới bắt đầu gửi dữ liệu.

Bộ kiểm tra sử dụng expected_mem[0:255] làm bộ nhớ tham chiếu, wr_idx/rd_idx làm chỉ số, biến pass/fail đếm kết quả so sánh. Task fifo_write gửi dữ liệu vào cạnh xuống wclk, task fifo_read kích hoạt đọc ở cạnh xuống rclk và kiểm tra sau khi dữ liệu đã được chốt qua cạnh lên.

2. Các kịch bản kiểm thử

Kịch bản	Dữ liệu/kích thích	Mục tiêu
Khởi tạo	wrst = rrst = 1 trong 100 ns	Con trỏ về 0; full = 0, empty = 1

Ghi liên tiếp	8 giá trị 0x10 đến 0x17	Kiểm tra đường ghi và trạng thái đầy
Chờ đồng bộ	#200 sau khi ghi	Cho con trỏ Gray truyền qua miền đọc
Đọc liên tiếp	8 lần gọi fifo_read	Kiểm tra thứ tự FIFO và dữ liệu ra
Kết thúc	#500 rồi in PASS/FAIL	Tổng kết số lượt so sánh

Kỳ vọng lý thuyết khi tám thao tác ghi/đọc đều được chấp nhận: dữ liệu ra theo thứ tự 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17. Không điền kết quả PASS/FAIL cụ thể trước khi chạy mô phỏng thực tế.

3. Phân tích hạn chế của testbench

Task fifo_write chỉ bật w_en nếu !full, nhưng luôn tăng wr_idx và ghi expected_mem sau cạnh lên wclk, kể cả khi yêu cầu không được chấp nhận. Tương tự, fifo_read chỉ bật r_en nếu !empty nhưng vẫn tăng rd_idx và kiểm tra dữ liệu, kể cả trường hợp FIFO chưa cho phép đọc. Vì vậy scoreboard hiện tại chưa bảo đảm phản ánh chính xác các giao dịch được DUT chấp nhận trong mọi kịch bản.

Ngoài ra, chuỗi kiểm thử hiện tại chỉ ghi đầy một lượt rồi đọc hết; chưa bao quát đọc/ghi đồng thời, reset giữa phiên, đổi tỉ lệ clock, nhiều lần vòng con trỏ, cố tình ghi khi full và đọc khi empty, cũng như kiểm tra riêng almost_full/almost_empty. Đây là các trường hợp nên bổ sung trước khi khẳng định thiết kế đã được kiểm chứng toàn diện.

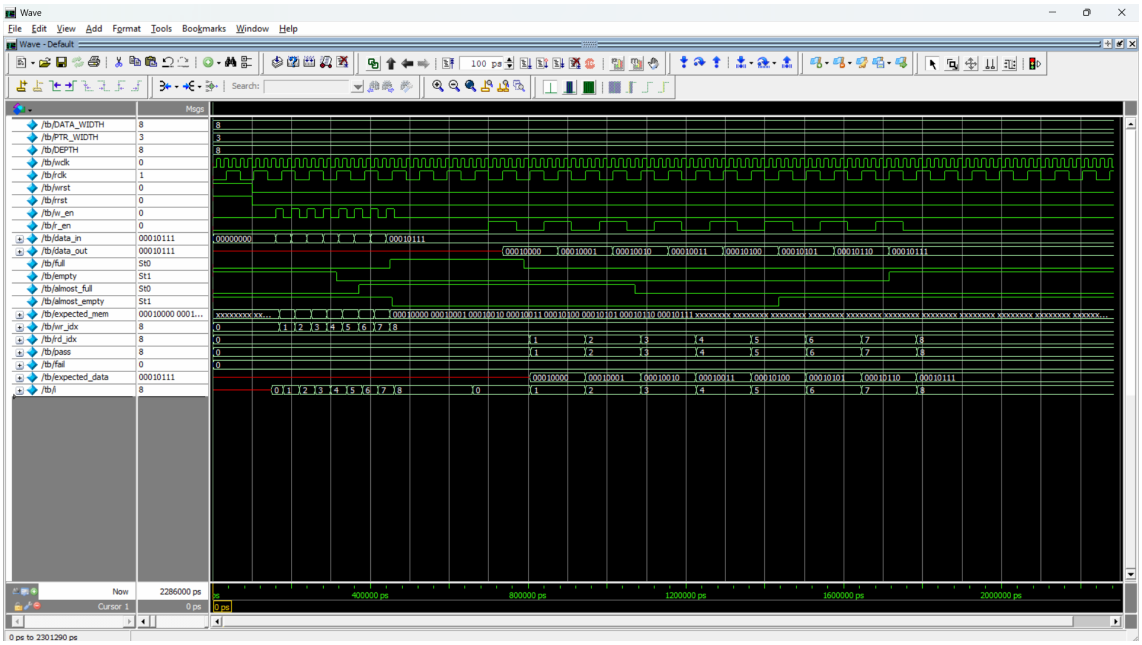
4. Hướng dẫn mô phỏng và ghi nhận kết quả

Có thể biên dịch các file theo thứ tự bất kỳ nếu công cụ hỗ trợ compile toàn bộ module cùng lúc. Ví dụ dùng Icarus Verilog:

```
iverilog -g2012 -s tb -o fifo_sim asynchronous_fifo_top.v fifo_mem.v synchronizer.v wptr_handler.v rptr_handler.v async_fifo_TB.sv

vvp fifo_sim
```

Đối với ModelSim/Questa: tạo thư viện work, biên dịch bằng vlog, sau đó chạy vsim work.tb; thêm tín hiệu wclk, rclk, w_en, r_en, full, empty, almost_full, almost_empty, data_in, data_out, b_wptr và b_rptr vào waveform. Lưu ảnh waveform và cửa sổ transcript để đưa vào báo cáo.



Hình 3. Waveform mô phỏng thực tế



Hình 4. Log kết quả chạy Self-checking Testbench (PASS = 8, FAIL = 0)

V. Kết Luận

Đề tài đã xây dựng cấu trúc RTL cho bộ nhớ FIFO bất đồng bộ với hai miền clock độc lập. Các khối chức năng được phân tách rõ ràng, gồm RAM hai cổng, hai bộ quản lý con trỏ và hai bộ đồng bộ Gray Code. Cơ chế kiểm soát full/empty sử dụng so sánh con trỏ Gray kế tiếp, trong khi cảnh báo almost_full/almost_empty dựa trên mức lấp đầy ước lượng từ con trỏ đã đồng bộ.

Thiết kế minh họa một phương pháp CDC thường gặp trong hệ thống FPGA/SoC. Hướng phát triển gồm: mở rộng testbench tự kiểm tra theo giao dịch thực tế; kiểm thử ngẫu nhiên với nhiều tỷ lệ clock; bổ sung assertion chống overflow/underflow; kiểm tra CDC/timing; tổng hợp trên FPGA và đo tài nguyên, tần số tối đa. Việc triển khai phần cứng chỉ nên báo cáo sau khi có bằng chứng thực nghiệm.

VI. Tài Liệu Tham Khảo

[1] Mã nguồn dự án Asynchronous FIFO: asynchronous_fifo_top.v, fifo_mem.v, synchronizer.v, wptr_handler.v, rptr_handler.v, async_fifo_TB.sv (tài liệu do người thực hiện cung cấp).

[2] README.md của dự án Asynchronous FIFO (tài liệu do người thực hiện cung cấp).

[3] Clifford E. Cummings, Simulation and Synthesis Techniques for Asynchronous FIFO Design, SNUG San Jose, 2002.

PHỤ LỤC – CÁC ĐOẠN RTL TIÊU BIỂU

1. Thuật toán chuyển đổi từ Binary sang mã Gray (Binary to Gray Conversion):
   wire [PTR_WIDTH:0] g_wptr_next = (b_wptr_next >> 1) ^ b_wptr_next;

2. Logic so sánh cờ Đầy (Full) và Rỗng (Empty) ở mã Gray:
   // Cờ Full (kiểm tra ở miền clock ghi):
   wire wfull = (g_wptr_next == {~g_rptr_sync[PTR_WIDTH:PTR_WIDTH-1], g_rptr_sync[PTR_WIDTH-2:0]});

   // Cờ Empty (kiểm tra ở miền clock đọc):
   wire rempty = (g_rptr_next == g_wptr_sync);

3. Cơ chế bảo vệ bộ nhớ (Chặn thao tác ghi/đọc khi tràn/cạn):
   .w_en(w_en && !full)
   .r_en(r_en && !empty)